

Exercício 2 - Coletar e Armazenar Dados do Cartola FC

Cenário:

Estamos iniciando um novo projeto de análise de dados focado em extrair insights e criar modelos preditivos sobre o desempenho dos jogadores e times no Cartola FC. Para isso, precisamos construir um repositório de dados robusto e atualizado. Nossa fonte de dados primária será a API não oficial do Cartola FC, que, apesar de não ser oficial, oferece acesso público aos dados do fantasy game. O nosso banco de dados de destino é o MongoDB, escolhido por sua flexibilidade em lidar com os dados semiestruturados (JSON) que a API fornece.

Seu Papel (Persona):

Neste projeto, você atuará como **Arquiteto(a) de Dados Python**. Sua responsabilidade será desenvolver os scripts em Python necessários para orquestrar todo o processo de ETL (Extração, Transformação e Carga): desde a requisição dos dados na API do Cartola FC até a sua gravação em nosso cluster MongoDB.

Objetivo Principal:

Desenvolver um conjunto de scripts em Python para automatizar a coleta de dados de *mercado*, *atletas* e *clubes* da API do Cartola FC e gravá-los de forma estruturada em coleções distintas no MongoDB. O script deve ser modular, robusto e preparado para ser agendado (por exemplo, via cron job) para execuções periódicas.

Requisitos Técnicos e Diretrizes:

1. Fonte de Dados (API):

- **API Base URL:** Utilize a API não oficial do Cartola FC. O principal endpoint que usaremos é o que consolida os dados de mercado:
 - <https://api.cartola.globo.com/atletas/mercado>
- **Análise da Resposta:** Antes de iniciar o código, analise a estrutura do JSON retornado por este endpoint. Identifique os principais objetos e arrays, como atletas, clubes, posicoes e status.

2. Ambiente de Desenvolvimento:

- **Linguagem:** Python 3.9+
- **Bibliotecas Essenciais:**
 - requests: Para realizar as chamadas HTTP à API.

- pymongo: Driver oficial para interagir com o MongoDB.
- python-dotenv (opcional, mas recomendado): Para gerenciar as credenciais do banco de dados de forma segura.

3. Banco de Dados de Destino (MongoDB):

- **Conexão:** As credenciais de conexão serão fornecidas separadamente (use um arquivo .env para armazená-las).
- **Database:** cartola_fc_db
- **Coleções a serem criadas:**
 1. mercado_rodada_atual: Para armazenar o status do mercado a cada coleta.
 2. atletas_rodada_atual: Para armazenar a lista completa de atletas e suas estatísticas na rodada atual.
 3. clubes_rodada_atual: Para armazenar as informações dos clubes.

4. Lógica do Script Python:

O script principal deve ser organizado da seguinte forma:

- **Módulo de Conexão:**
 - Crie uma função conectar_mongodb() que lê as credenciais do ambiente e estabelece a conexão com o MongoDB, retornando o objeto do banco de dados (db).
- **Módulo de Extração:**
 - Crie uma função buscar_dados_mercado() que faz uma requisição GET para a URL da API.
 - Implemente um tratamento de erros robusto: verifique o status code da resposta (e.g., response.raise_for_status()). Em caso de falha, o script deve registrar o erro e parar a execução de forma controlada.
 - A função deve retornar o conteúdo JSON da resposta decodificado.
- **Módulo de Transformação e Carga (ETL Core):**
 - Desenvolva uma função principal processar_e_gravar_dados(db, dados_mercado).
 - Dentro desta função, separe os dados recebidos do JSON principal:
 - **Dados dos Clubes:**

- Extraia o objeto clubes.
 - Para cada clube, transforme o documento para o formato desejado (exemplo abaixo).
 - Utilize a operação `pymongo.operations.UpdateOne` com `upsert=True` para inserir ou atualizar os clubes na coleção `clubes_rodada_atual`, usando o id do clube como chave de busca. Isso evita duplicatas.
 - **Dados dos Atletas:**
 - Extraia a lista atletas.
 - Adicione um campo `timestamp_coleta` a cada documento de atleta para sabermos quando o dado foi capturado.
 - Antes de inserir, limpe a coleção `atletas_rodada_atual` (`db.atletas_rodada_atual.delete_many({})`) para garantir que apenas os dados da última consulta estarão presentes.
 - Insira a lista completa de atletas na coleção `atletas_rodada_atual` usando `insert_many()`.
 - **Dados do Status do Mercado:**
 - Extraia o objeto status.
 - Adicione um campo `timestamp_coleta`.
 - Insira este único documento na coleção `mercado_rodada_atual`. Considere limpar a coleção antes, se desejar manter apenas o status mais recente.
 - **Orquestração (main):**
 - Crie um bloco `if __name__ == "__main__":` para orquestrar a execução.
 - Chame a função de conexão com o MongoDB.
 - Chame a função para buscar os dados da API.
 - Passe os dados obtidos para a função de processamento e gravação.
 - Inclua logs (print statements ou a biblioteca logging) para indicar cada etapa do processo: "Conectando ao MongoDB...", "Buscando dados na API...", "Gravando dados dos clubes...", "Finalizado com sucesso."
-

Exemplo da Estrutura de Documentos no MongoDB:

Coleção: clubes_rodada_atual

JSON

```
{
  "_id": 293, // Usar o ID do clube como _id para facilitar o upsert
  "nome": "Fluminense",
  "abreviacao": "FLU",
  "escudos": {
    "60x60":
      "https://s.sde.globo.com/media/organizations/2014/04/14/fluminense_60x60.png",
    "45x45":
      "https://s.sde.globo.com/media/organizations/2014/04/14/fluminense_45x45.png",
    "30x30":
      "https://s.sde.globo.com/media/organizations/2014/04/14/fluminense_30x30.png"
  },
  "nome_fantasia": "Fluminense"
}
```

Coleção: atletas_rodada_atual

JSON

```
{
  "atleta_id": 37624,
  "nome": "Fábio",
  "apelido": "Fábio",
  "foto":
    "https://s.sde.globo.com/media/person_role/2023/04/28/fabio_220x220.png",
  "preco_num": 5.33,
  "variacao_num": -0.42,
  "clube_id": 262,
  "posicao_id": 1, // Goleiro
}
```

```
"status_id": 7, // Provável
"jogos_num": 15,
"pontos_num": 3.4,
"media_num": 2.22,
"scout": {
  "g": 0,
  "a": 0,
  "ds": 51,
  "sg": 3
  // ... outros scouts
},
"timestamp_coleta": "2024-08-14T12:00:00Z" // Padrão ISO 8601
}
```

Coleção: mercado_rodada_atual

JSON

```
{
  "rodada_atual": 19,
  "status_mercado": 1, // 1 para Aberto, 2 para Fechado
  "aviso": "",
  "fechamento": {
    "dia": 17,
    "mes": 8,
    "ano": 2024,
    "hora": 15,
    "minuto": 0,
    "timestamp": 1723917600
  },
  "timestamp_coleta": "2024-08-14T12:00:00Z"
}
```

}